

SERVIÇO PÚBLICO FEDERAL · MINISTÉRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE VIÇOSA · UFV
CAMPUS FLORESTAL



ATIVIDADE JUNIORS POO S7

PEDRO ELECIO FERNANDES
REALINO - 5786

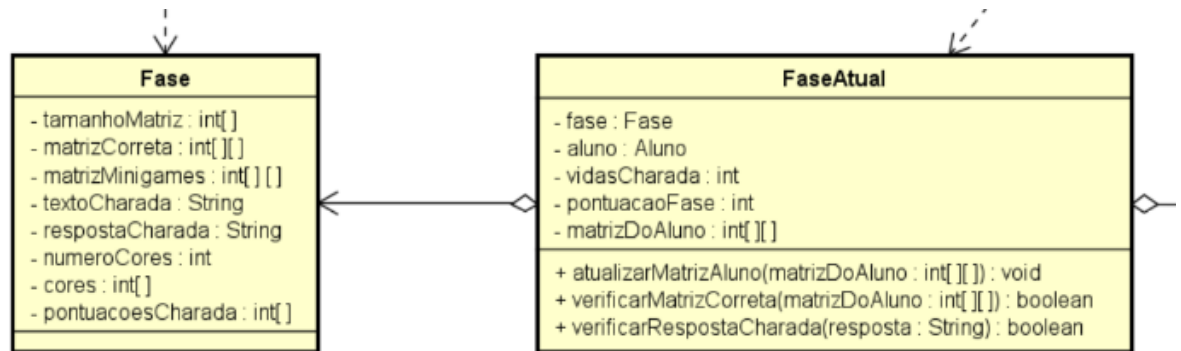
Florestal - MG

Outubro de

2025

Introdução

O objetivo desta atividade é implementar duas novas classes sendo elas, a classe “Fase” que conterá todas as regras da fase para futuras comparações de regra e a classe “FaseAtual” que implementa as lógicas de verificação das regras da fase (resposta da charada, matriz pintada corretamente ..). Essa classe também tem alguns métodos e atributos, que serão explicados mais abaixo.



Parte I – Implementação da Classe Fase:

Nesta parte será necessário a criação e implementação de uma classe, os atributos necessários estão abaixo:

Classe Fase:

Classe que criará sempre um objeto estático que será imutável. Ela conterá todas as regras da fase para futuras comparações de regras, e para passagem inicial das mesmas ao frontend.

1. Atributos:

- tamanhoMatriz (privado , tipo: vetor de Inteiro)
- matrizCorreta (privado , tipo: vetor de Inteiro)
- matrizMinigames (privado , tipo: matriz de inteiro)
- textoCharada (privado , tipo: String)
- respostaCharada (privado , tipo: String)
- numeroCores (privado , tipo: Inteiro)
- cores (privado , tipo: vetor de Inteiro)

Parte II – Implementação da Classe FaseAtual:

Nesta parte será necessário a criação e implementação de outra classe, os atributos necessários estão abaixo:

Classe FaseAtual:

1. Atributos:

- fase (privado , tipo: Fase)
- aluno (privado , tipo: Aluno)
- vidasCharada (privado , tipo: Inteiro)
- pontuacaoFase (privado , tipo: Inteiro)
- matrizDoAluno (privado , tipo: matriz de Inteiro)

2. Métodos:

- **atualizarMatrizAluno**: ao finalizar o desenho, o frontend envia ao backend a matriz construída, salvando então na matrizDoAluno.

- **verificarMatrizCorreta:** após a atualização da matriz do jogador, o método recebe a matriz atualizada e compara com a matriz correta implementada em Fase. Retornando verdadeiro ou falso, para permitir que o aluno avance para a próxima etapa da fase.
- **verificarRespostaCharada:** recebe a resposta do aluno para a charada e realiza a comparação com a resposta correta implementada em Fase. Retornando verdadeiro ou falso, para o número de vidas restantes e a pontuação serem atualizados, e o frontend mostrar que a resposta foi correta ou incorreta.

Parte III – Envio

O código desenvolvido deverá ser enviado via GitHub. Com base nos tutoriais disponibilizados, vocês deverão criar um *pull request* para que a entrega da atividade seja analisada.

Uma branch deve ser criada por cada um dos Juniors a partir da branch develop, utilizando o seguinte formato:

“junior/nome_Junior/POO_S7”

O título do pull request deverá ser padrão, ele será: “Junior: Cria classe Fase & Cria classe FaseAtual” .

Após a criação da branch os arquivos fontes Fase.java e FaseAtual.java criados, devem ser comitado dentro do diretório “juniors/sprint_7/POO” , na branch respectiva de cada junior, e então abrir um pull request deve ser aberto, seguindo o padrão projeto.

Coloquem como reviewer das atividades de POO o “El3ci0Sz”.

Não esqueçam de seguir o padrão de commit e pull requests, especificando a milestone, labels. E também mover as tarefas no projects.